

Nama : Muhammad Husain Kaasyiful Ghitha

NIM : 1103220047

Kelas TK-45-G09

Robotika dan Sistem Cerdas

Analisis Tugas Robotika Week 15

Pada tugas ini, dilakukan praktik simulasi robot menggunakan Robot Operating System (ROS) Noetic Ninjemys di Ubuntu 20.04 LTS. Praktik dilakukan berdasarkan pedoman yang disediakan di buku [Mastering ROS for Robotics Programming](#), yang memiliki kode sumber yang disediakan secara terbuka di [GitHub](#). Selain itu, juga terdapat petunjuk pelaksanaan simulasi dari [Code in Action](#) yang mempermudah pelaksanaan praktik ini.

Dari delapan bab yang menjadi target pengerjaan tugas ini, enam di antaranya terselesaikan dengan baik. Berikut adalah keenam bab yang dikerjakan:

- Introduction to ROS
- Getting Started with ROS Programming
- Working with ROS for 3D Modeling
- Simulating Robots Using ROS and Gazebo
- Simulating Robots Using ROS, CoppeliaSim, and Webots
- Using the ROS MoveIt! and Navigation Stack

Berikut adalah pembahasan mengenai simulasi robot yang dilakukan, termasuk dari perintah, konfigurasi, hingga visualisasi dan simulasi yang dihasilkan:

1. Introduction to ROS.

Di buku tersebut, disebutkan bahwa tatacara penginstalan dapat dilihat di situs wiki resmi ROS, yang memberikan penjelasan detail penggunaan untuk setiap distribusi ROS. Pada wiki resmi ROS, langkah pertama penginstalan adalah memastikan repositori Ubuntu agar mengizinkan kategori "restricted," "universe," dan "multiverse." Setelah itu, tambahkan repositori ROS ke sistem dengan menjalankan perintah `sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc)`

main" > /etc/apt/sources.list.d/ros-latest.list'. Langkah ini memungkinkan sistem mengakses paket-paket ROS yang diperlukan untuk instalasi. Pastikan repositori ini telah diatur dengan benar sebelum melanjutkan ke langkah berikutnya.

Selanjutnya, tambahkan kunci keamanan agar sistem dapat memverifikasi paket dari repositori ROS. Jika curl belum terpasang di sistem, instal terlebih dahulu menggunakan 'sudo apt install curl'. Kemudian, tambahkan kunci keamanan dengan menjalankan 'curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -'. Proses ini memastikan bahwa semua paket yang diunduh dari repositori adalah asli dan aman.

Setelah repositori dan kunci keamanan disiapkan, perbarui indeks paket Debian dengan perintah 'sudo apt update'. Setelah itu, pilih jenis instalasi ROS yang sesuai dengan kebutuhan. Instalasi "Desktop-Full" untuk semua fitur lengkap (termasuk simulator 2D/3D dan paket persepsi), "Desktop" untuk alat-alat seperti rqt dan rviz tanpa simulator, atau "ROS-Base" untuk instalasi minimal tanpa antarmuka grafis. Untuk menginstal fitur tertentu, paket individu seperti 'sudo apt install ros-noetic-slam-mapping' dapat diinstal secara terpisah. Pada praktik ini, instalasi yang dipilih adalah "Desktop-Full", dengan beberapa tambahan di bab selanjutnya.

Agar ROS dapat digunakan di terminal, lingkungan ROS harus diatur dengan menjalankan 'source /opt/ros/noetic/setup.bash'. Untuk kenyamanan, tambahkan perintah ini ke file ~/.bashrc menggunakan 'echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc' sehingga akan dijalankan secara otomatis setiap kali membuka terminal baru. Langkah ini penting untuk memastikan bahwa ROS terintegrasi dengan shell.

Terakhir, instal alat tambahan seperti rosinstall yang dibutuhkan untuk membangun dan mengelola proyek ROS dengan perintah 'sudo apt install python3-rosdep python3-roscpp python3-roscpp-msgs python3-roscpp-msgs-generate python3-wstool build-essential'. Inisialisasi rosdep dengan menjalankan sudo rosdep init diikuti oleh rosdep update. Setelah menjalankan seluruh command yang ada di petunjuk instalasi, verifikasi penginstalan dapat dilakukan dengan menjalankan roscore, yang merupakan master dari node-node yang ada di ROS.

2. Getting Started with ROS Programming.

Untuk dapat menggunakan ROS, langkah pertama adalah memastikan bahwa workspace telah disiapkan dengan benar. Dari panduan yang diberikan, repository kode sumber "Mastering ROS for Robotics Programming" dapat di-clone dari GitHub

menggunakan perintah 'git clone'. Setelah proses clone selesai, pengguna berpindah ke direktori project dan membuat workspace baru bernama 'catkin_ws' di dalam home directory. Workspace ini kemudian dibangun menggunakan perintah 'catkin_make', yang menghasilkan struktur folder yang diperlukan untuk pengembangan ROS, seperti 'devel', 'build', dan 'src'.

Selanjutnya, paket demo dari repository dipindahkan ke dalam folder 'src' di workspace, dan workspace dibangun kembali dengan perintah 'catkin_make'. Setelah itu, environment setup dilakukan dengan menambahkan perintah 'source ~/catkin_ws/devel/setup.bash' ke dalam file '~/.bashrc', sehingga ROS dapat terintegrasi otomatis saat terminal baru dibuka. Langkah ini memastikan bahwa workspace ROS yang baru dibuat siap digunakan.

Setelah workspace siap, pengguna menjalankan berbagai node ROS untuk eksplorasi. Misalnya, menjalankan 'roscore' adalah langkah wajib untuk memulai master ROS, yang bertindak sebagai pengatur komunikasi antar-node. Perintah seperti 'roslaunch mastering_ros_demo_package demo_topic_publisher' dan 'demo_topic_subscriber' digunakan untuk mempublikasikan dan berlangganan topik tertentu. Output yang dihasilkan mencakup urutan bilangan yang dikirimkan antar-node melalui topik yang aktif. Sama halnya seperti pertukaran pesan yang dilakukan dengan perintah 'roslaunch mastering_ros_demo_package demo_msg_publisher' dan 'demo_msg_subscriber'.

Pada langkah selanjutnya, pengguna dapat mencoba menjalankan server dan klien layanan dengan perintah seperti 'roslaunch mastering_ros_demo_pkg demo_service_server' dan 'demo_service_client'. Server layanan ini akan mendengarkan permintaan dari klien dan mengirimkan respons sesuai dengan implementasi yang diberikan. Hal serupa berlaku untuk eksplorasi server dan klien aksi, di mana klien dapat mengirimkan tugas berjangka panjang ke server dan memantau kemajuan tugas tersebut, mendeteksi hal-hal yang menghambat pengerjaan tugas.

Terakhir, pengguna dapat memanfaatkan file peluncuran ('launch file') untuk menjalankan beberapa node secara bersamaan menggunakan perintah 'roslaunch mastering_ros_demo_pkg demo_topic.launch'. Setelah file ini dijalankan, daftar node aktif dapat dilihat dengan perintah 'roslaunch mastering_ros_demo_pkg demo_topic.launch', yang memberikan informasi tentang node-node yang sedang berjalan dalam sistem ROS. Output dari langkah-langkah ini mencerminkan fungsi dan komunikasi antar-node, serta memberikan pengalaman langsung dalam mengembangkan aplikasi berbasis ROS.

3. Working with ROS for 3D Modeling.

Urutan command yang ada pada Bab 3 buku Mastering ROS for Robotics Programming digunakan untuk mempersiapkan dan memvisualisasikan model robot 3D menggunakan ROS. Langkah pertama adalah mengatur lingkungan ROS agar workspace yang dibuat otomatis dimuat setiap terminal baru dibuka, dengan menambahkan perintah `source ~/catkin_ws/devel/setup.bash` ke dalam file `~/.bashrc`. Selanjutnya, kode sumber proyek diunduh dan file dari folder Chapter3 yang berasal dari repository kode sumber "Mastering ROS for Robotics Programming", yang berisi deskripsi model robot, disalin ke folder `src` dalam workspace `catkin_ws`.

Setelah file tersedia di dalam workspace, perintah `catkin_make` dijalankan untuk membangun (compile) workspace. Proses ini menghasilkan folder `build` dan `devel` yang diperlukan untuk menjalankan proyek ROS. Dengan workspace yang telah dibangun, pengguna dapat meluncurkan file deskripsi robot menggunakan beberapa perintah `roslaunch`. Misalnya, perintah `view_demo.launch` digunakan untuk menampilkan model robot demo yang memperagakan konfigurasi sederhana dari sebuah robot berupa perputaran dan pemiringan, `view_arm.launch` untuk model manipulator robot (lengan) dengan tujuh derajat kebebasan, dan `view_mobile_robot.launch` untuk robot mobile sederhana dengan konfigurasi differential drive.

Hasil dari perintah ini adalah tampilan model 3D robot di RViz. Visualisasi ini menunjukkan detail robot, termasuk properti fisik seperti sendi (joint) dan penghubung (link), serta elemen visual seperti material dan kolisi. Dengan memanfaatkan RViz, pengguna dapat memeriksa dan memodifikasi model 3D robot sebelum menggunakannya lebih lanjut dalam simulasi atau implementasi nyata. Untuk meninjau dan memodifikasi model robot secara langsung, konfigurasi dari robot-robot tersebut dapat dilihat di file `'urdf'`, yang berisikan file-file bertipe `.urdf` (Universal Robot Description Format) dan file-file `.xacro`. file-file ini berisikan konfigurasi seperti bentuk elemen-elemen yang menyusun suatu robot, collision box, dan dependensi yang diperlukan dalam membangun robot.

4. Simulating Robots Using ROS and Gazebo.

Bab 4 dari buku Mastering ROS for Robotics Programming membahas simulasi robot menggunakan ROS dan Gazebo. Script ini dimulai dengan mengunduh atau

memindahkan file dari folder Chapter4 dan juga Chapter6 ke dalam folder src di workspace catkin_ws. Beberapa perintah dari bagian ini memerlukan kode yang terletak di folder 'Chapter6' pada repository. Langkah ini diikuti dengan menjalankan perintah catkin_make untuk membangun workspace ROS, yang diperlukan untuk mengintegrasikan file yang telah dipindahkan. Workspace ini mencakup paket-paket untuk simulasi lengan robot dengan tujuh derajat kebebasan (seven degrees of freedom, 7-DOF) dan robot beroda diferensial.

Untuk mensimulasikan lengan robot 7-DOF, file peluncuran seven_dof_arm_gazebo_control.launch digunakan. Perintah ini memuat model lengan robot ke dalam simulator Gazebo dan mengaktifkan kontroler yang memungkinkan pergerakan sendi (joint). Dengan menggunakan perintah rostopic pub, posisi sendi robot dapat dikendalikan secara manual. Sebagai contoh, perintah rostopic pub /seven_dof_arm/joint4_position_controller/command std_msgs/Float64 "data: 1.0" mengatur posisi sendi keempat lengan robot ke 1.0 radian. Output dari langkah ini adalah visualisasi di Gazebo, di mana lengan robot dapat dilihat bergerak sesuai perintah.

Untuk robot beroda diferensial, simulasi diluncurkan menggunakan diff_wheeled_gazebo_full.launch, yang memuat model robot beroda ke dalam Gazebo lengkap dengan sensor-sensor seperti lidar dan kamera. Selanjutnya, perintah keyboard_teleop.launch memungkinkan pengguna untuk mengendalikan robot secara manual menggunakan input dari keyboard. Output dari proses ini adalah simulasi robot beroda di Gazebo yang dapat bergerak sesuai dengan input kontrol. Seperti contoh, menekan tombol 'i' pada node keyboard_teleop akan membua robot bergerak maju ke depan.

Dengan memanfaatkan Gazebo sebagai simulator, script ini memberikan lingkungan yang baik untuk menguji kontroler, pergerakan, dan interaksi robot dengan lingkungannya, sehingga mempermudah pengembangan robot sebelum diimplementasikan pada perangkat nyata.

5. Simulating Robots Using ROS, CoppeliaSim, and Webots.

Bab ini membahas simulasi robot menggunakan ROS bersama dengan dua simulator populer, yaitu CoppeliaSim dan Webots. Script ini dimulai dengan instalasi manual CoppeliaSim Edu melalui ekstraksi file '.tar.xz' dan konfigurasi variabel lingkungan 'COPPELIASIM_ROOT' untuk akses mudah ke direktori utama

CoppeliaSim. File dari 'Chapter5' kemudian dipindahkan ke folder 'src' di dalam workspace ROS ('catkin_ws') dan workspace dibangun menggunakan 'catkin_make'. Untuk menjalankan simulasi di CoppeliaSim, perintah './coppeliaSim.sh' memulai aplikasi, dan plugin terkait seperti 'plugin_publisher_subscriber.ttt' atau model robot seperti 'seven_dof_arm.ttt' dimuat. Pengguna dapat berinteraksi dengan simulator menggunakan perintah 'rostopic'. Sebagai contoh, perintah 'rostopic pub /csim_demo/seven_dof_arm/elbow_pitch/cmd std_msgs/Float32 "data: 1.0"' mengontrol posisi sendi lengan robot (elbow_pitch) ke sudut 1.0. Untuk memantau perubahan, perintah 'rostopic echo' dapat digunakan, yang akan menampilkan data real-time dari posisi sendi tersebut. Selain itu, 'rqt_image_view' memungkinkan pengguna memvisualisasikan data kamera yang dihasilkan dari simulasi.

Selanjutnya, untuk simulasi menggunakan Webots, aplikasi Webots diluncurkan melalui perintah 'webots' setelah 'roscore' diaktifkan. File dunia ('world') seperti 'e_puck_ros.wbt' dimuat untuk mensimulasikan robot e-puck. Perintah 'rostopic echo /model_name' menampilkan nama model aktif di simulasi, sedangkan perintah 'rosservice call' mengaktifkan layanan kamera robot untuk menangkap data visual. Simulasi ini juga mendukung kontrol manual robot menggunakan perintah 'keyboard_teleop.launch', yang memerlukan manajer kontrol khusus seperti 'e_puck_manager'.

Di CoppeliaSim, hasil yang didapat dari simulasi 'plugin_publisher_subscriber.ttt' adalah sepasang objek yang terdiri atas objek pemantau yang memiliki kamera dan objek yang bergerak secara memutar untuk dipantau. Meskipun terdapat tampilan kamera di CoppeliaSim, masukan mentah kamera dapat dilihat melalui 'rqt_image_view'. Pada simulasi 'seven_dof_arm.ttt', output yang dihasilkan adalah gerakan sendi robot lengan 7-DOF yang sesuai dengan perintah dan data status yang terus diperbarui secara real-time di terminal. Visualisasi dari kamera robot, jika digunakan, dapat diakses di GUI ROS atau aplikasi eksternal.

Sementara itu, di Webots, output berupa data dari sensor, layanan kamera aktif, dan gerakan robot E-puck yang dapat diamati dalam simulasi. Integrasi ini menunjukkan bagaimana ROS memungkinkan kontrol dan komunikasi antara simulator dan sistem robotik.

6. Using the ROS MoveIt! and Navigation Stack

Bab 6 buku *Mastering ROS for Robotics Programming* menjelaskan integrasi ROS MoveIt! dan Navigation Stack untuk simulasi manipulasi dan navigasi robot. Script ini dimulai dengan menyalin file dari folder-folder pada setiap chapter hingga chapter ini ke dalam folder src di workspace ROS (catkin_ws) dan menjalankan catkin_make untuk membangun workspace. Setelah itu, paket-paket utama seperti ros-noetic-moveit dan ros-noetic-navigation diinstal untuk mendukung manipulasi robot dan navigasi.

Untuk manipulasi robot lengan 7-DOF, MoveIt! digunakan dengan memulai MoveIt! Setup Assistant melalui perintah `'roslaunch moveit_setup_assistant setup_assistant.launch'`. Ini memungkinkan pengguna untuk membuat dan mengonfigurasi paket manipulasi robot, termasuk mendefinisikan grup pergerakan dan titik referensi. Paket yang dihasilkan diuji menggunakan `demo.launch`, yang memuat model robot dan antarmuka manipulasi di RViz. Perintah seperti `'roslaunch seven_dof_arm_bringup_moveit.launch'` digunakan untuk menghubungkan simulasi di Gazebo dengan kontroler MoveIt!, sehingga memungkinkan kontrol lengan robot secara interaktif.

Untuk navigasi robot beroda diferensial, Navigation Stack diaktifkan dengan perintah seperti `'roslaunch diff_wheeled_robot_gazebo gmapping.launch'` untuk membangun peta lingkungan secara simultan menggunakan sensor robot. Setelah peta selesai dibuat, perintah `map_saver` digunakan untuk menyimpan peta ke file, yang kemudian dapat digunakan untuk navigasi otonom menggunakan Adaptive Monte Carlo Localization (AMCL). Perintah `'roslaunch diff_wheeled_robot_gazebo amcl.launch'` memuat algoritma lokalisasi untuk memungkinkan robot menentukan posisinya dalam peta. Perintah kontrol `'roslaunch diff_wheeled_robot_control keyboard_teleop.launch'` memungkinkan kontrol manual robot menggunakan keyboard, sedangkan automasi pergerakan dengan AMCL dilakukan dengan memanfaatkan peta dan rute yang telah disiapkan.

Output dari simulasi manipulasi menunjukkan kemampuan robot lengan untuk melakukan pergerakan terencana, baik melalui antarmuka RViz maupun kontrol manual. Hasilnya adalah model robot yang dapat berinteraksi dengan lingkungannya secara akurat. Untuk navigasi, robot beroda menghasilkan peta lingkungan yang terlihat di RViz, sementara pergerakannya dapat diamati langsung di simulasi Gazebo. Dengan AMCL, robot mampu bernavigasi secara otonom berdasarkan peta yang telah disimpan,

menunjukkan kemampuan penuh dari sistem navigasi berbasis ROS. Script ini menciptakan lingkungan yang komprehensif untuk pengembangan manipulasi dan navigasi robot otomatis.